

第1章 微分方程式の数値計算と散逸則 (2026年度)

散逸則を持つ常微分方程式を例にとり，数値計算法と散逸則との関係を見る．

1.1 常微分方程式

次の常微分方程式を考える：

$$\begin{cases} x'(t) = -f_x(x(t)), & t > 0 \\ x(0) = x_0 \in \mathbb{R} \end{cases} \quad (\text{ODE})$$

ここで， $x = x(t) : [0, \infty) \rightarrow \mathbb{R}$ は未知関数， $f = f(x) : \mathbb{R} \rightarrow \mathbb{R}$ は与えられた非負値関数， $x_0 \in \mathbb{R}$ は与えられた初期値である．

(ODE) の解に対して，次の散逸則が成り立つ．

命題 1.1 (散逸則 (微分型)). $x = x(t)$ を (ODE) の解とすると，すべての $t > 0$ に対して

$$\frac{d}{dt} f(x(t)) = -|f_x(x(t))|^2 \quad (1.1)$$

が成り立つ．

証明 合成関数の微分法と (ODE) より，

$$\frac{d}{dt} f(x(t)) = f_x(x(t)) \frac{dx}{dt}(t) = f_x(x(t)) \cdot (-f_x(x(t))) = -|f_x(x(t))|^2$$

が得られる．

□

(1.1) を $0 \leq t \leq T$ で積分すると，次の積分型の散逸則がわかる：

$$f(x(T)) = f(x(0)) - \int_0^T |f_x(x(t))|^2 dt \quad (1.2)$$

1.2 常微分方程式の計算例

f を具体的に与えて微分方程式 (ODE) の解を求めてみる．

例 1.1. $f(x) = \frac{1}{2}x^2$ とすると， $f_x(x) = x$ となるから，(ODE) は

$$\begin{cases} x'(t) = -x(t), & t > 0 \\ x(0) = x_0 \in \mathbb{R} \end{cases}$$

となる。この解は $x(t) = x_0 e^{-t}$ である。

例 1.2. $f(x) = \frac{1}{4}x^4$ とすると, $f_x(x) = x^3$ となるから, (ODE) は

$$\begin{cases} x'(t) = -(x(t))^3, & t > 0 \\ x(0) = x_0 \in \mathbb{R} \end{cases}$$

となる。これは陽的に解けて

$$x(t) = \begin{cases} \frac{1}{\sqrt{2t+x_0^2}} & (x_0 > 0) \\ 0 & (x_0 = 0) \\ -\frac{1}{\sqrt{2t+x_0^2}} & (x_0 < 0) \end{cases}$$

となる。

例 1.3. $f(x) = 1 + \cos x$ とすると, $f_x(x) = -\sin x$ となるから, (ODE) は

$$\begin{cases} x'(t) = \sin(x(t)), & t > 0 \\ x(0) = x_0 \in \mathbb{R} \end{cases}$$

となる。これは陽的に解けて

$$x(t) = \begin{cases} \frac{1}{\sqrt{2t+x_0^2}} & (x_0 > 0) \\ 0 & (x_0 = 0) \\ -\frac{1}{\sqrt{2t+x_0^2}} & (x_0 < 0) \end{cases}$$

となる。この解は $x(t) = 2 \arctan(\tan(x_0/2)e^t)$ である。

例 1.4. $f(x) = e^{x^2}$ とすると, $f_x(x) = 2e^{x^2}$ となるから, (ODE) は

$$\begin{cases} x'(t) = -2(x(t))e^{(x(t))^2}, & t > 0 \\ x(0) = x_0 \in \mathbb{R} \end{cases}$$

となる。これは陽的に解けないらしい。

1.3 数値的に求める

一般的に (ODE) の解は陽的に書けないことの方が多い。そこで、微分を差分におきかえて、近似解を探すことを試みる。

微分係数の定義

$$x'(t) = \lim_{\Delta t \rightarrow 0} \frac{x(t + \Delta t) - x(t)}{\Delta t}$$

において、十分小さい時間刻み幅 $\Delta t > 0$ を固定して、

$$x'(t) \approx \frac{x(t + \Delta t) - x(t)}{\Delta t} \tag{1.3}$$

と近似すると, (ODE) は次のように差分化できる:

$$\frac{x(t + \Delta t) - x(t)}{\Delta t} = -f_x(x(t))$$

これを $x(t + \Delta t)$ について解いて

$$x(t + \Delta t) = x(t) - \Delta t f_x(x(t))$$

としよう. 時間ステップを $t = 0, \Delta t, 2\Delta t, 3\Delta t, \dots$ と進めると,

$$x(\Delta t) = x(0) - \Delta t f_x(x(0)),$$

$$x(2\Delta t) = x(\Delta t) - \Delta t f_x(x(\Delta t)),$$

$$x(3\Delta t) = x(2\Delta t) - \Delta t f_x(x(2\Delta t)),$$

となる. $a_n = x(n\Delta t)$ ($n = 0, 1, 2, \dots$) とおくと, 次の漸化式を解く問題に変形できる:

$$\begin{cases} a_{n+1} = a_n - \Delta t f_x(a_n), & n = 0, 1, 2, \dots \\ a_0 = x_0 \end{cases} \quad (\text{RE})$$

目的の時間 $T > 0$ における値 $x(T)$ を求めたいときは, $N\Delta t \geq T$ となるようなステップ数 N (すなわち $N \geq \frac{T}{\Delta t}$) を決めればよい.

1.4 漸化式をみたす数列を求めるスキーム

(RE) をみたす数列を求めるスキームの構成は以下の通りとなる.

与える定数

- 初期値 $x_0 \in \mathbb{R}$
- 時間刻み幅 $\Delta t > 0$
- 最終時間 $T > 0$

ここから, 総ステップ数 N を

$$N = \text{切り上げ} \left(\frac{T}{\Delta t} \right)$$

で求める.

与える関数

- 非負値関数 $f: \mathbb{R} \rightarrow \mathbb{R}$

ここから, f の導関数 f_x を計算する.

注意. f は与えられた関数だから, その導関数 f_x は厳密に計算できる. 従って, (1.3) のような近似計算は必要ない. しかしながら, 導関数を求める計算は「記号計算」と呼ばれるもので, 「数値計算」とは異なる方法をとる (mathematica も記号計算ができる). そこで, python のプログラミングでは, 「記号計算」ができるライブラリである sympy を使って, 与えられた f から微分を「記号計算」で計算させる. その後, sympy の lambdify 関数を使って, 「記号計算」として定めた関数を「数値計算」に利用できるように変換するようにした.

初期化

numpy 型の配列 $a_0, a_1, a_2, \dots, a_N$ を $a_0 = a_1 = a_2 = \dots = a_N = 0.0$ で初期化する. そのあと, $a_0 = x_0$ と置く.

繰返計算 (漸化式を求める)

$n = 0, 1, 2, \dots, N-1$ について, 以下の代入文を繰り返し実行する:

$$a_{n+1} = a_n - \Delta t f_x(a_n).$$

1.5 近似解で散逸則は成り立つか

上記で得られた $a_N \approx x(T)$ に対して, 積分型の散逸則 (1.2) が成り立つかどうかを調べる. そのために, 積分については台形則を用いることにする. すなわち

$$\begin{aligned} & \int_0^T |f_x(x(t))|^2 dt \\ &= \frac{1}{2} |f_x(x(0))|^2 \Delta t + |f_x(x(\Delta t))|^2 \Delta t + |f_x(x(2\Delta t))|^2 \Delta t + \dots + |f_x(x((N-1)\Delta t))|^2 \Delta t + \frac{1}{2} |f_x(x(N\Delta t))|^2 \Delta t \\ &= \left(|f_x(x(0))|^2, |f_x(x(\Delta t))|^2, |f_x(x(2\Delta t))|^2, \dots, |f_x(x(N\Delta t))|^2 \right) \cdot \left(\frac{1}{2}, 1, 1, \dots, 1, \frac{1}{2} \right) \Delta t \end{aligned}$$

を用いることにする. (1.2) が成り立つかどうかをみるために,

$$\begin{aligned} & \left| f(x(T)) - \left(f(x(0)) - \int_0^T |f_x(x(t))|^2 dt \right) \right| \\ &= \left| f(a_N) - \left(f(a_0) - \left(|f_x(a_0)|^2, |f_x(a_1)|^2, |f_x(a_2)|^2, \dots, |f_x(a_N)|^2 \right) \cdot \left(\frac{1}{2}, 1, 1, \dots, 1, \frac{1}{2} \right) \Delta t \right) \right| \end{aligned}$$

を計算させてみよう.

1.6 アルゴリズムの詳細と実装

本稿で用いた, 常微分方程式 (ODE) の差分法 (前進オイラー法) による数値シミュレーション, およびエネルギー関数の時間変化 (台形公式による積分) を検証するための具体的な計算

手順を以下に示す。

1.6.1 計算アルゴリズム

時間ステップ幅を Δt 、最終時刻を T とし、反復回数を $N = \lceil T/\Delta t \rceil$ とする。初期値 x_0 から出発し、関数 $f(x)$ に対する速度場 $v(x) = -\frac{df}{dx}$ を用いて状態 $a_n \approx x(n\Delta t)$ を更新する。また、エネルギーの保存 (減少) 誤差を台形公式による数値積分を用いて評価する。

Algorithm 1 前進オイラー法による軌道計算とエネルギー変化の検証

Input: 初期値 x_0 , 時間刻み幅 Δt , 最終時刻 T , 評価対象の関数 $f(x)$

Output: 状態の履歴 $\{a_n\}_{n=0}^N$, エネルギー誤差 $\mathcal{E}$

```
1:  $N \leftarrow \lceil T/\Delta t \rceil$  { 総反復回数の決定 }
2:  $v(x) \leftarrow -\frac{df}{dx}$  { 速度関数 (導関数) の定義 }
3: 配列  $a$  をサイズ  $N+1$  で初期化
4:  $a_0 \leftarrow x_0$  { 初期条件の設定 }
5: for  $n = 0$  to  $N-1$  do
6:    $a_{n+1} \leftarrow a_n + \Delta t \cdot v(a_n)$  { 前進オイラー法による更新 }
7: end for
8: 各時刻  $t_n = n \cdot \Delta t$  ( $n = 0, \dots, N$ ) に対する時間配列  $\{t_n\}$  を生成
9:  $y_n \leftarrow |v(a_n)|^2$  ( $n = 0, \dots, N$ ) { 速度の 2 乗の計算 }
10:  $I \leftarrow \Delta t \left( \frac{1}{2}y_0 + \sum_{n=1}^{N-1} y_n + \frac{1}{2}y_N \right)$  { 台形公式による  $\int_0^T |v(x(t))|^2 dt$  の近似計算 }
11:  $\mathcal{E} \leftarrow |f(a_N) - f(0) + I|$  { エネルギー誤差の計算 }
12: return  $\{a_n\}_{n=0}^N, \mathcal{E}$ 
```

1.6.2 プログラムの構成と処理の流れ

上記のアルゴリズムを以下の 3 つのフェーズで実装した。

- シンボリック計算 (**SymPy** による自動微分) : 評価したい関数 $f(x) = \exp(x^2)$ を定義し、その導関数の -1 倍である $v(x) = -2x \exp(x^2)$ を自動微分により算出する。その後、`lambdify` を用いて高速な数値計算 (**NumPy**) が可能な形式へと変換する。
- 時間発展のシミュレーション (**NumPy**) : 前進オイラー法 (Forward Euler Method) に基づき、漸化式 $a_{n+1} = a_n + \Delta t \cdot v(a_n)$ をループ処理によって繰り返し計算し、状態変数 $x(t)$ の時系列データを取得する。
- エネルギー解析と可視化 (**Matplotlib**) : 得られた軌道から、各時刻におけるエネルギー $f(x(t))$ の減衰挙動をプロットする。さらに、エネルギーの減少量と散逸項の積分値 (台形公式で計算) を比較し、理論解からの数値的な乖離を表す「エネルギー誤差」を出力する。

1.7 コメント

常微分方程式の数値計算については，[1,3,4] を参考にせよ．本稿でのソースコードは `python` を用いた．ソースコード中のライブラリや関数については，[2] を参考にせよ．

参考文献

- [1]E. (Ernst) Hairer, G. Wanner, and 蟹江 幸博, 解析教程 (上), シュプリンガー・フェアラーク東京, 1997.
- [2]かくあき, 現場で使える!python 科学技術計算入門 : *Numpy/sympy/scipy/pandas* による数値計算・データ処理手 法, AI & technology, 翔泳社, 2020.
- [3]橋本 修 and 毛塚 敦, *Python* による数値計算法の基礎, 森北出版, 2021.
- [4]堀之内 総一, 酒井 幸吉, and 榎園 茂, *C* による数値計算法入門, 第 2 版, 森北出版, 2015.